

Ex.No.10 Use Ghidra to disassemble and analyze the malware code.

Date :

Project Objectives

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
- To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
- To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md

- Overview: Brief description of Ghidra, its installation, and how it is used for malware analysis.
- Requirements: List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
- Project Outline: A breakdown of the tutorial steps, linking to various sections or files within the repository.

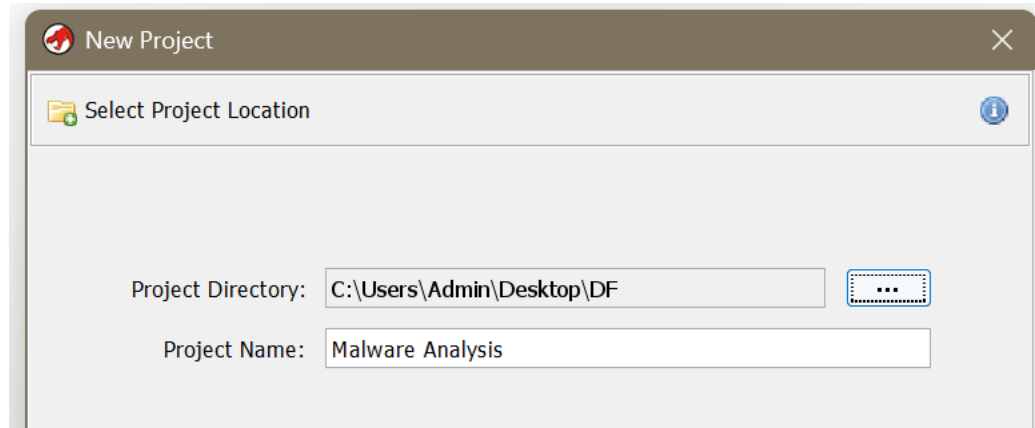
2. Tutorial Steps (Markdown or Jupyter Notebooks)

- Step-by-step instructions, broken down into digestible sections, such as:
 - Environment Setup: Instructions on setting up a virtualized and isolated environment.
 - Initial Analysis: How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - Function Analysis: Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - String and Import Analysis: How to locate and interpret strings and imports related to malicious behavior.
 - Advanced Techniques: Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.

3. Example Scripts (Python or Java)

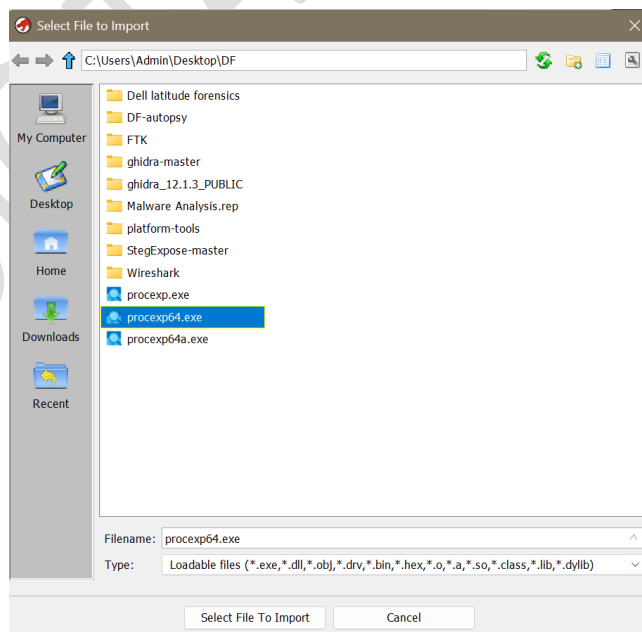
- Scripts to automate repetitive analysis tasks, such as:
 - Extracting and listing all string references.
 - Identifying network-related functions or common indicators of malware.

- Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).
- **Script Examples:** Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).



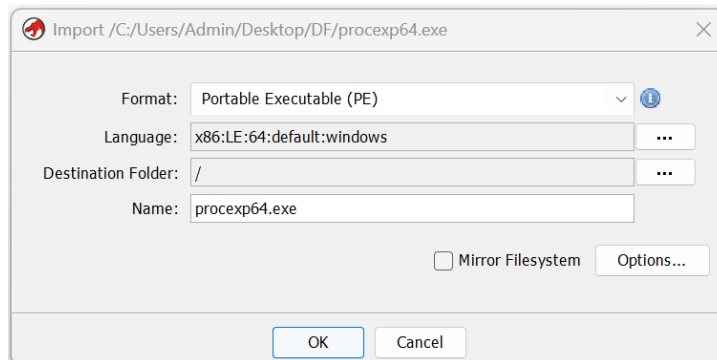
4. Sample Data (Benign Binary or Hex Dump)

- Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.
- Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - Executing system calls.
 - Writing to files.
 - Sending network data.
- Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.



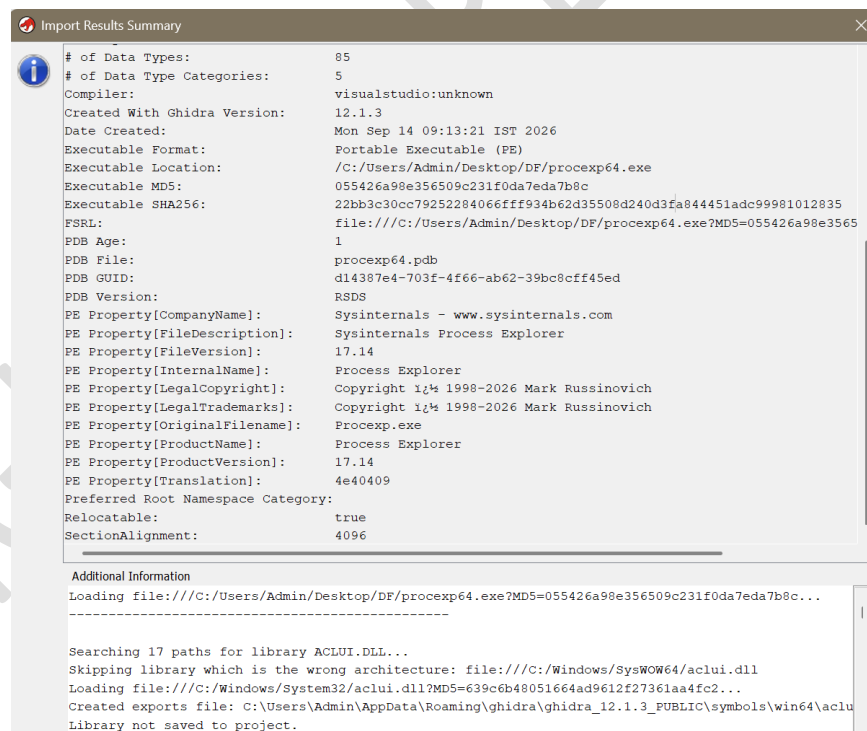
5. Documentation and Comments

- Include explanations of the code within each script, detailing what each function does.
- Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.



6. Report Template

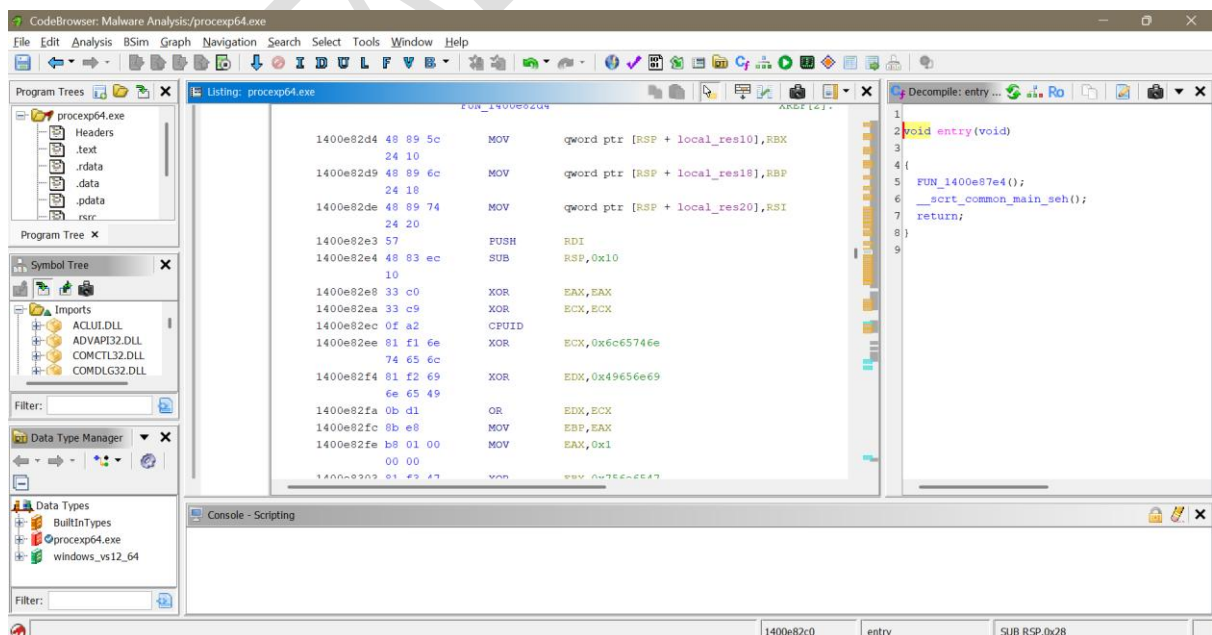
- A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:
 - Summary of Analysis: Brief description of the suspected malware.
 - Function Analysis: Summary of the key functions, their purpose, and the findings.



- Behavioral Indicators: Summary of malware behaviors, such as persistence, file system manipulation, or network communications.
- Recommendations: Suggestions on further actions, such as containment, eradication, or further analysis.

Example Repository Structure

```
plaintext
ghidra-malware-analysis-project/
├── README.md
├── docs/
│   ├── Tutorial_Step1.md
│   ├── Tutorial_Step2.md
│   └── Tutorial_StepN.md
├── scripts/
│   ├── extract_strings.py
│   ├── network_analysis.py
│   └── label_functions.py
├── samples/
│   ├── benign_sample1.bin
│   └── benign_sample2.hex
├── templates/
├── analysis_report_template.md
├── screenshots/
├── function_graph_example.png
└── decompiler_example.png
```



Project Development and Future Work

- **Expand Scripts:** Continue adding Ghidra scripts to automate complex analysis tasks.
- **Community Contributions:** Encourage community members to contribute additional benign samples, analysis scripts, and documentation improvements.
- **Advanced Topics:** Include guides for advanced techniques, such as unpacking obfuscated code, identifying encryption routines, and using the P-Code emulator for runtime analysis.

Rubrics

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	